



evosoft Hungary Kft. Óbudai Egyetem

Nagy rendszerek fejlesztésének technológiája
Domain modell

Unrestricted © evosoft Hungary Kft. 2026

www.evosoft.com

01

Szoftverfejlesztési
módszertanok

02

Modellek

03

Domain modell

04

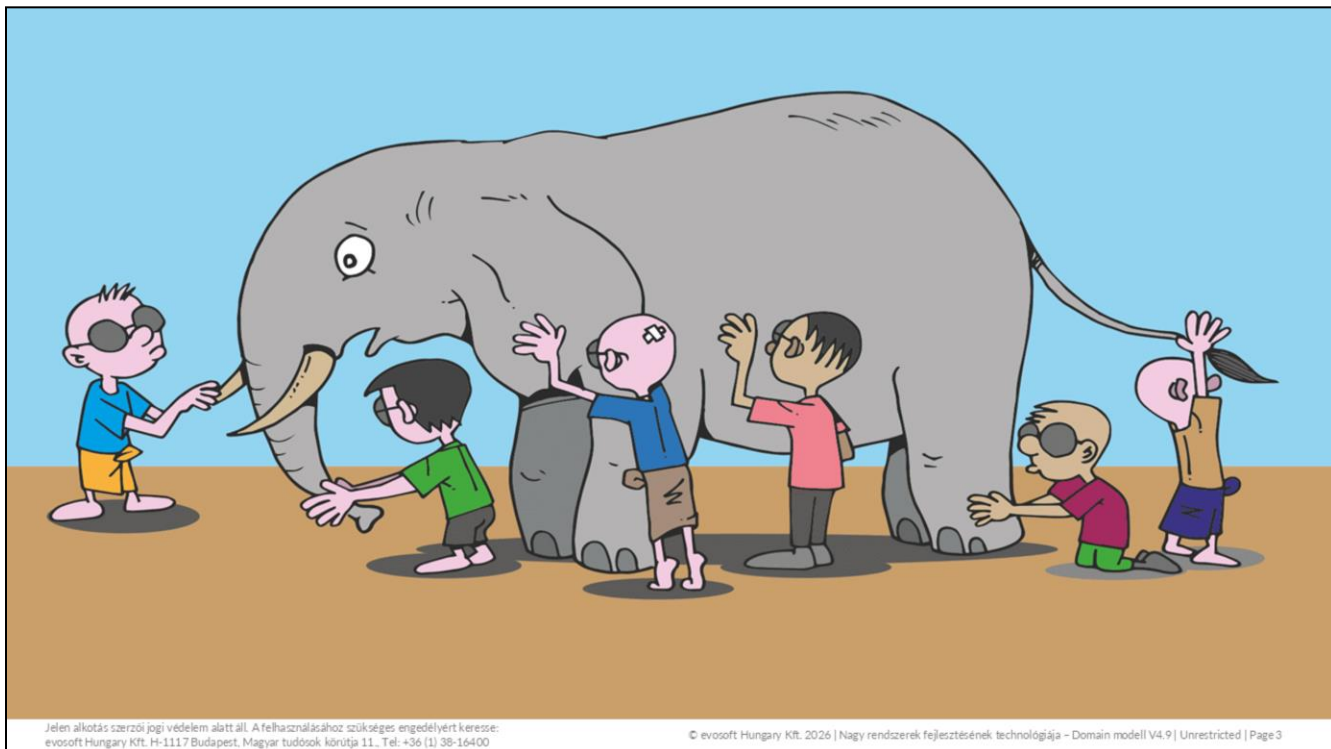
Domain-vezérelt tervezés

05

CRUD és CQRS

06

Összefoglalás



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája - Domain modell V4.9 | Unrestricted | Page 3

Élt egyszer régen egy kis faluban hat vak ember. Nem látták soha a napvilágot, így a tárgyakról, élőlényekről, a körülöttük lévő világról is csak többi érzékszervük alapján alakították ki saját képüket. Egy napon egy elefántot hoztak eléjük, és megkérdezték, hogyan is nézhet ki szerintük. Odamentek hát a hatalmas állathoz, és mindannyian megérintették.

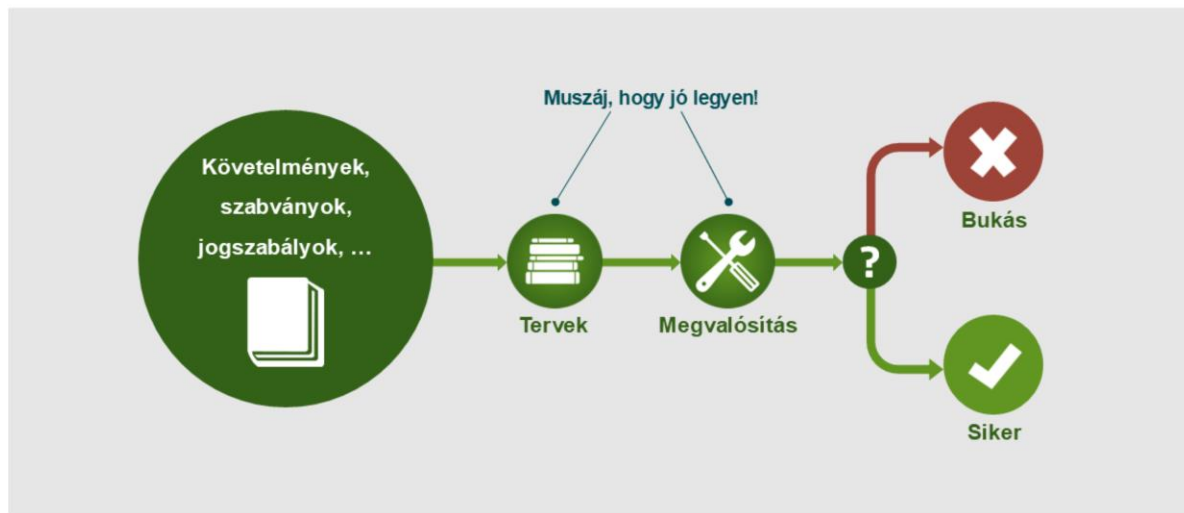
- „Nahát! Az elefánt olyan, mint egy vastag faág.” – szólalt meg egyikük, aki az agyarát tapogatta.
- „Nem, nem, inkább olyan, mint egy cső.” – mondta a másik, aki épp az ormányát fogta.
- „Szerintem inkább olyan, mint egy hatalmas legyező.” – szólt a harmadik, aki az állat fülét fogdosta.
- „Nincs igazatok, az elefánt úgy néz ki, mint egy nagy fal.” – állapította meg a negyedik, miközben az elefánt oldalát tapogatta.
- „Dehogy! Az elefánt olyan, mint egy oszlop.” – szólalt meg az ötödik vak ember, aki az állat lábát tapogatta meg.
- „Szerintem viszont mindannyian tévedtek, mert az elefánt olyan, mint egy köté!.” – vélte a hatodik, aki a farkát fogta.

Nagy szoftverrendszerek fejlesztésében mindig sokan vesznek részt, ahol az egyes szereplők mind nagyon sokfajta nézőponttal, képességekkel és tudással rendelkeznek. Ezen nézőpontokat egy egységes egészé kell alakítani.

Szoftverfejlesztési módszertanok (és problémáik)

Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 4

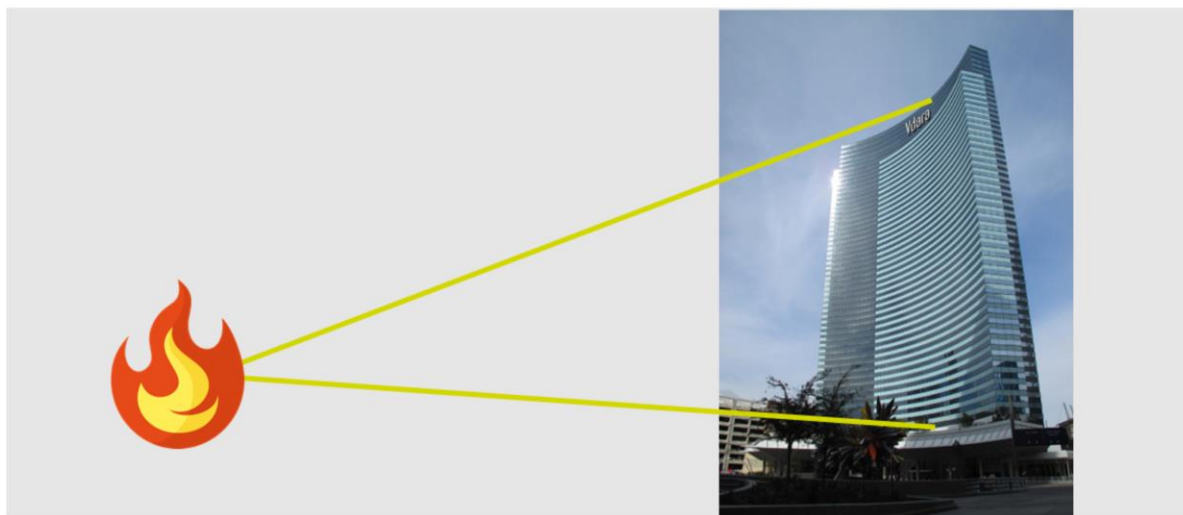


Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája - Domain modell V4.9 | Unrestricted | Page 5

A klasszikus szoftverfejlesztési modellt ezzel a folyamattal tudnánk leírni:

- Az elején megkapjuk a szoftverhez tartozó összes kapcsolódó anyagot: a kontextust, a követelményeket, a megkötéseket.
- Ezek alapján el kell készíteni a részletes szoftver design-t, amely a teljes rendszert leírja.
- A fejlesztők a tervek alapján elkészítik az implementációt.
- Ha szerencsénk van, és a terv jól sikerült, valamint a fejlesztés nem tért el a tervektől, akkor a projekt sikeres.
- Az esetek túlnyomó részében viszont a terv nem sikerül tökéletesre, vagy a fejlesztés tér el végül a tervektől, és a projekt megbukik.



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 6

A fenti Las Vegas-i hotel tervezésekor például nem gondoltak arra, hogy a parabolikus kialakítás miatt az ablaküvegek a szomszédos hotel medencéjére fogják majd fókuszálni a napfényt, ami nem kevés kellemetlenséget okozott. A problémát csak nagyon költségesen tudták megoldani, az ablakok nem tükröződő fóliával való bevonásával.

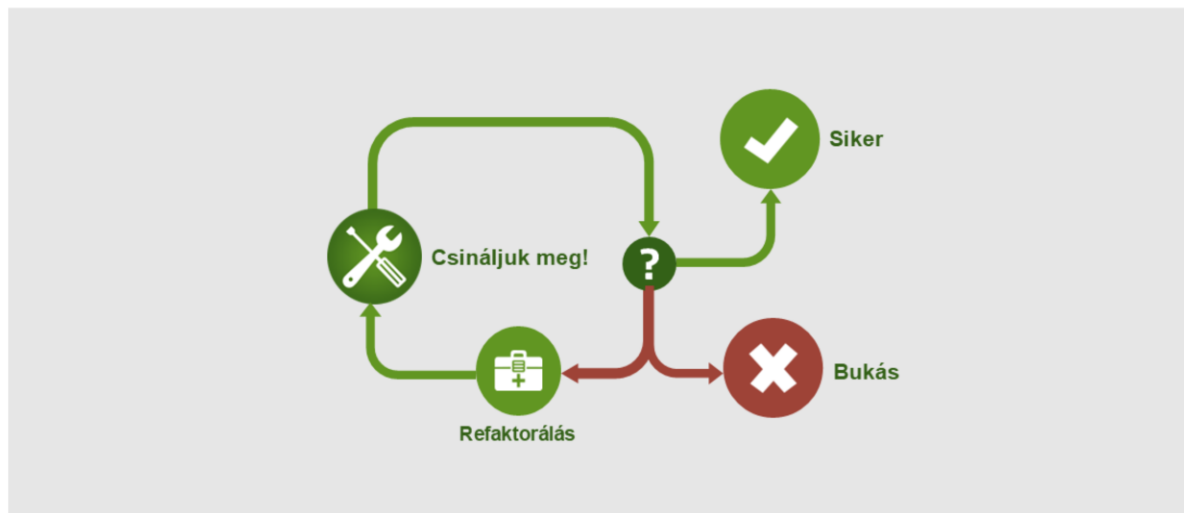


Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája - Domain modell V4.9 | Unrestricted | Page 7

A klasszikus szoftverfejlesztési megközelítéssel több alapvető probléma is van.

- Mivel a terveknek tökéletesnek kell lenniük, így a tervezők könnyen **analízis paralízis**be eshetnek, azaz nem tudnak a tervezési fázisból továbblépni, mert a modell még mindig „nem elég jó”. Ez a folyamat elhúzódását, ezáltal közvetlen veszteséget és piacvesztést okozhat, valamint megnöveli az esélyét annak, hogy a tervek - mire végre elkészülnek - már tényleg nem lesznek jók.
- A sok terv rendkívül rugalmatlan, **merev rendszert** eredményez, amely nem kezeli jól a változásokat. Az elkerülhetetlen változtatások esetén pedig gyakran „borul minden”.
- Az eredmény nagyon gyakran a sok terv miatt egy **komplex szoftver** lesz, amelyet nehéz átlátni, nehéz bővíteni, nehéz módosítani és nehéz karbantartani.



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

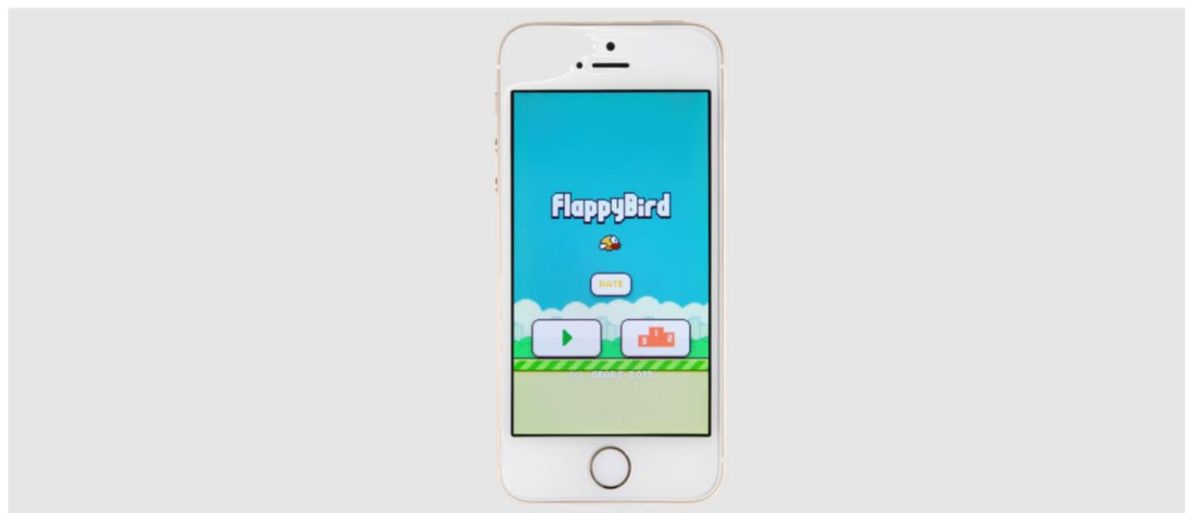
© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája - Domain modell V4.9 | Unrestricted | Page 8

A nagy tervektől való félelem kialakított egy sokkal egyszerűbb, „tervmentes” megközelítést is a szoftverfejlesztésben. Ebben egyszerűen nekilátunk a fejlesztésnek, ezáltal gyorsan lehet valami konkrét eredményt mutatni a megrendelőnek, aki így lehetőséget kap arra, hogy korán tudjon reagálni az elképzeléseinkre. Ha a bemutatott megoldás nem felel meg valamilyen szempontból, akkor újra nekifutunk, átalakítjuk, refaktoráljuk a rendszert, és egy rövid ciklus után újra bemutatjuk.

A gyors visszacsatolás, a gyors folyamatok és a gyors konkrét eredmény miatt ez a módszer meglehetősen népszerű a startupok és a kis cégek világában. Szerencsés esetben a startupot ilyenkor hamar felvásárolja egy nagyobb multi cég, ami a startup szempontjából egyértelmű sikernek számít.

Sajnos a startupok többsége nem ilyen szerencsés, és ha hosszabb ideig dolgoznak ezzel a megközelítéssel, akkor a bukás előbb-utóbb elkerülhetetlen lesz. A rendszer ugyanis idővel annyira kaotikussá válik, hogy a refaktorálás egyre nehezebb lesz, és hamarosan eljön az a pillanat, amikor már nem éri meg tovább foltoztatni a rendszert.

Egy sikeres példa – de ez ritka!

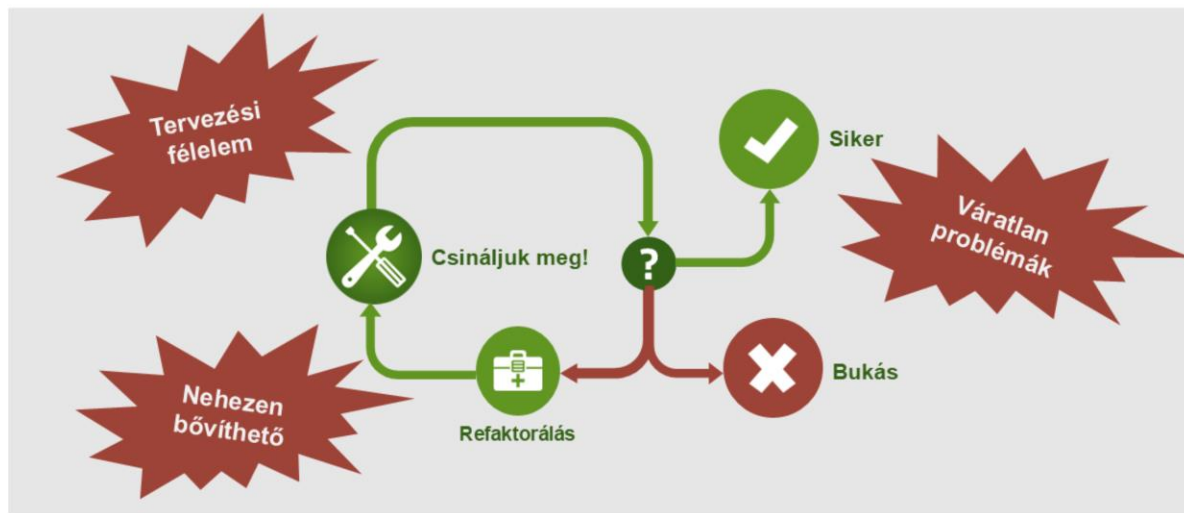


Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 9

Egyik ilyen ismertebb példa a Flappy Birds telefonos játék – itt nem volt semmi design, a fejlesztő néhány nap alatt készítette el a játékot, mégis óriási sikere volt. Nem készült megvalósíthatósági tanulmány, architektúra specifikáció, dinamikus modellek... De ilyen siker több millióból egyszer fordul elő.

Ráadásul hamar kiderült, hogy a játék sikere a rendkívül addiktív, de idegesítő játékmenetnek köszönhető, ami néhány hét alatt felháborodásba csapott át, ami miatt a fejlesztő törölte is a játékot a piacterekről – kétségtelen, hogy addigra már jelentős haszonra tett szert a rövid fejlesztésből.

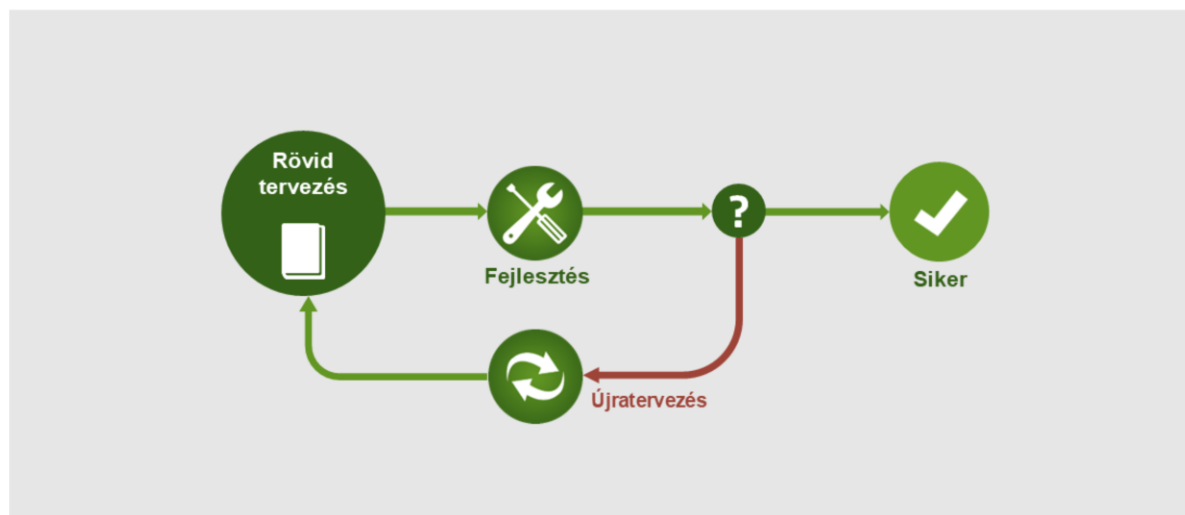


Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 10

A tipikus problémák az egyszerű fejlesztési folyamattal:

- **Félelem a tervezéstől**, ami sokszor éppen a nagy rendszereknél tapasztalt túlzott tervezésekből, illetve az ebből fakadó bénultságból ered.
- A gyorsan összedobott rendszerek rövid idő alatt eljutnak abba a fázisba, amikor már nagyon **nehezen bővíthető**, vagy egyáltalán nem lehet őket bővíteni.
- Mivel nem készülünk előre, így sokszor bukkannak elő **váratlan problémák**, amelyekre aztán egyre nehezebb reagálni.

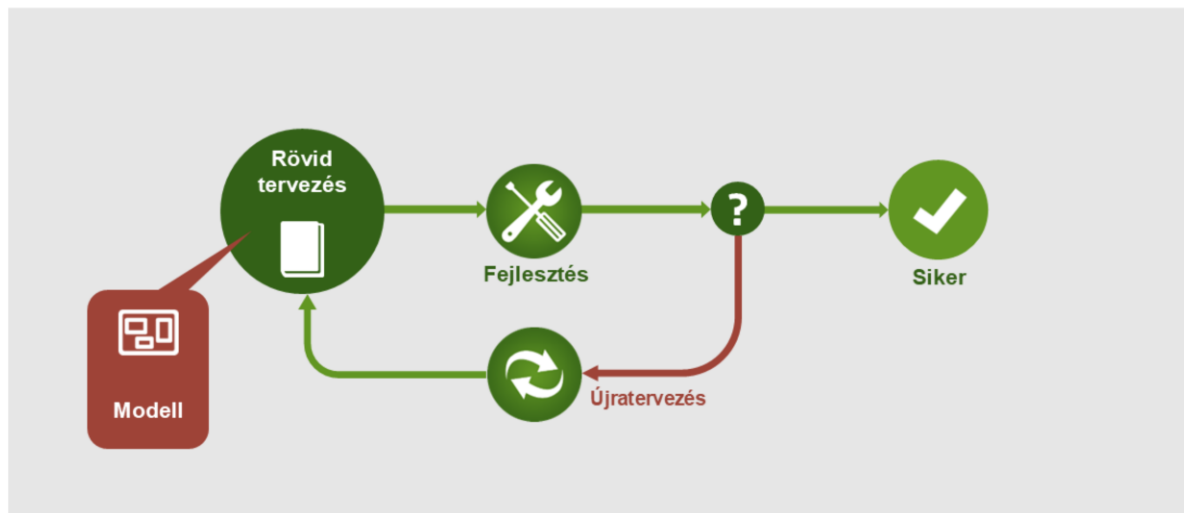


Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 11

A jó megoldás a szoftverfejlesztésre a két véglet között van, amikor végzünk tervezést, de csak annyit, amennyire éppen szükség van. Ez a megközelítés illeszkedik legjobban az agilis módszerekhez.

- Folyamatosan működőképes.
- Folyamatosan legyen benne valamilyen érték.
- Ne legyen túl bonyolult – lehessen könnyen változtatni.
- Ne készüljünk a jövőre túl előre, mert úgyis változni fog.



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 12

A tervezés egyik leghasznosabb eleme a **modell** elkészítése, frissítése.

A modell biztosítja, hogy valóban megértettük a problémát, a feladatot, és ez alapján tudunk egyeztetni az elvárásokról is a megrendelővel - legyen az egy számítógépes mérnök, vagy akár egy banki szakember.

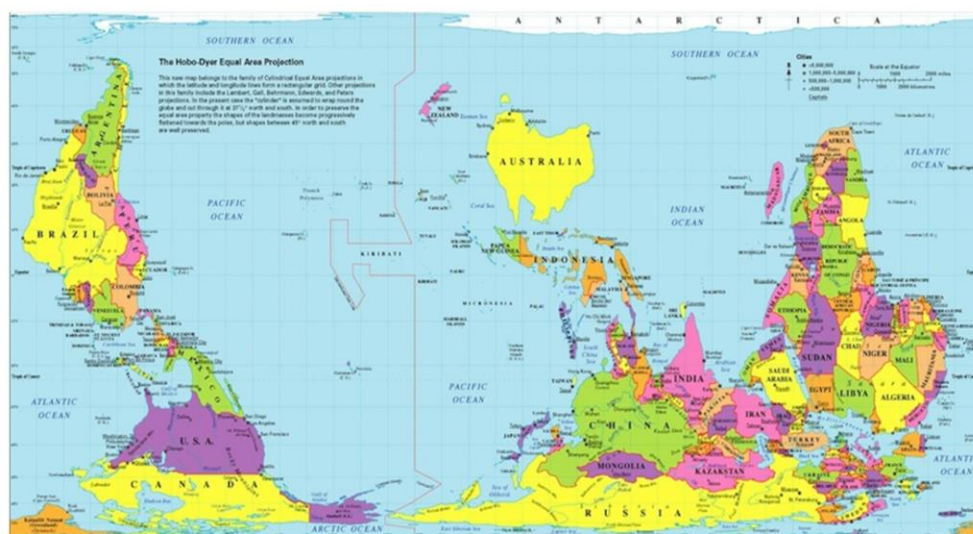
Modellek



“Lényegében
minden modell rossz,
de van, amelyik hasznos.”

George E. P. Box

A modellek, mint olyanok, nem tökéletesek. Minden modellre igaz, hogy valójában rossz, **nem fedi le a valóságot**, viszont ha jól csináljuk, akkor hasznos lesz. Mivel a modellek nem kell tökéletesek legyenek, így a modell megalkotásakor arra kell fókuszálni, hogy a modellt használni tudjuk (**legyen hasznos**), nem pedig arra, hogy minél tökéletesebb legyen.



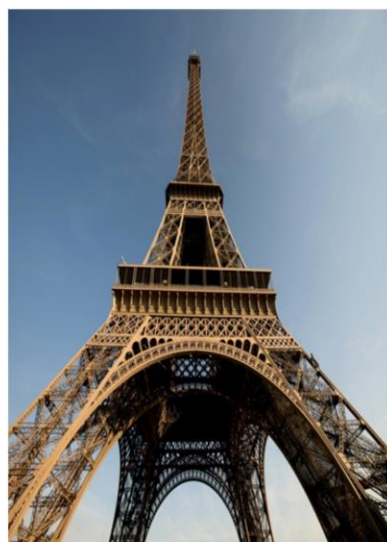
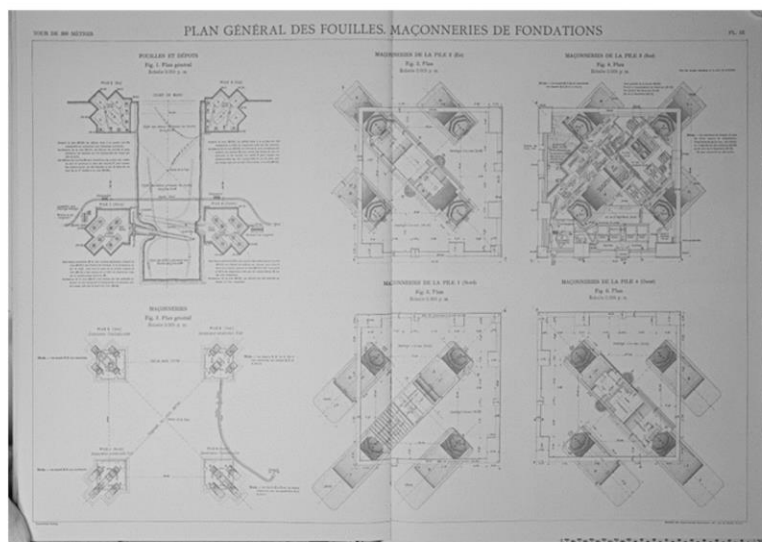
Jelen alkotás szerzői jogi védelem alatt áll. A felhasználásához szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája - Domain modell V4.9 | Unrestricted | Page 15

A térképészet egy olyan területe a tudománynak, amely közösmert modelleket gyárt. Ezek a térképek, amelyek a Föld felületének ábrázolásai.

Az általunk általánosan használt világtérképeket úgynevezett Mercator vetítéssel készítették. Ezeknek viszont van egy nagy hibája: a sarkok felé egyre jobban torzíja a szárazföldek területét.

Ezzel szemben az itt látható Hobo-Dyer vetítésű térkép célja, hogy a szárazföldek valós méretét mutassa – cserébe az alakjuk fog eltérni a műholdas felvételeken látható valótól. Nézzünk egy szemléletes példát. A Mercator vetítésű térképeken Grönland területe közelítőleg megegyezik Afrikáéval. Pedig Grönland a valóságban 14-szer kisebb! Itt láthatjuk is a valós viszonyokat.



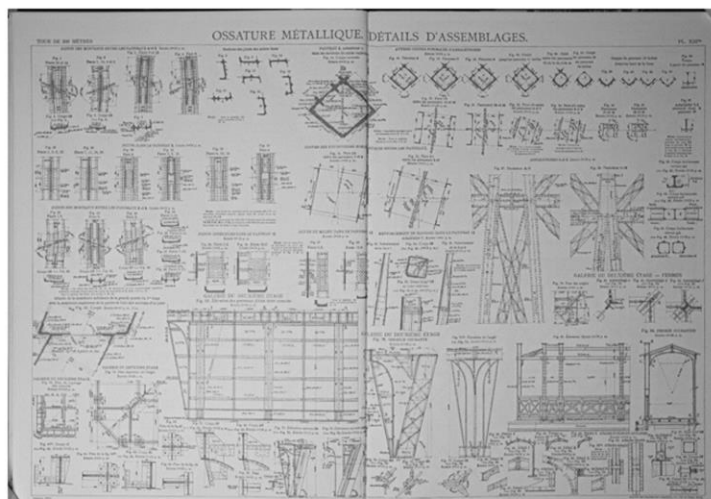
Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 16

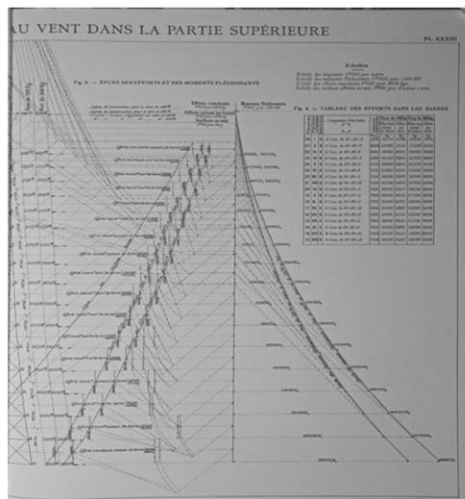
A Wikipedia oldalán elérhetően a párizsi Eiffel torony eredeti tervrajzai, amelyek meglepő hasonlóságokat mutatnak a szoftveres tervrajzokkal.

Link: https://commons.wikimedia.org/wiki/Category:Eiffel_Tower_plans

Részletes terv



Dinamikus nézet



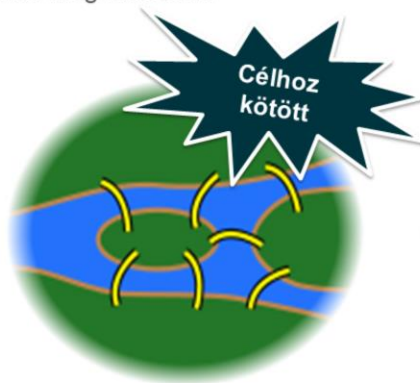
Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 17

Néhány példa az Eiffel torony terveiből: egy **részletes modell** a torony különböző részeihez, valamint egy **dinamikus nézet**, amely a torony viselkedését írja le erős szélben.

Látható, hogy a kis részletekből milyen nehéz meglátni a nagy egészet.

A **modell** egy rendszer egyszerűsítése, egy konkrét célhoz kötve. Segít a rendszerben felmerült kérdéseket megválaszolni.



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse: evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

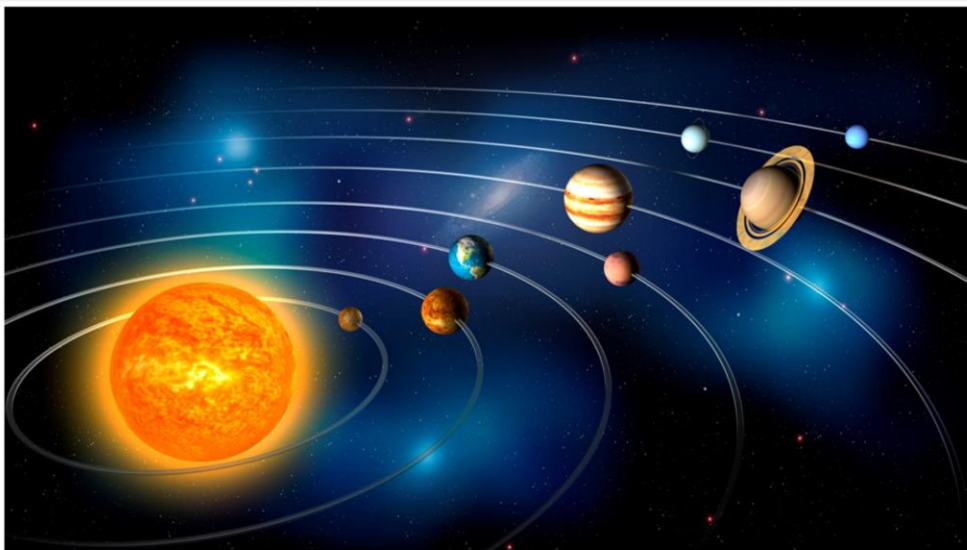
© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 18

A modellekre egy példa a Königsbergi hidak problémája. Euler azt gyanította, hogy a hét különböző hídon nem lehet úgy végigmenni, hogy az ember mindegyik hidat pontosan egyszer érinti. A próbálkozások a hidakon problémásak és sokáig tartanak. Egy egyszerű modellel könnyebben végig lehet gondolni és megérteni a problémát. A modell további egyszerűsítésével (és nem a bonyolításával!) még inkább a kérdésre, a problémára lehet fókuszálni - ezáltal a megoldást is könnyebben megtalálhatjuk. Euler végül az egyszerű modell alapján megalkotta a gráfelmélet alapjait.

A modell legfontosabb tulajdonságai:

- **Egyszerűsít** - az eredeti problémát egyszerűbb, érthetőbb formában mutatja meg, elhagyva a fölösleges részeket.
- **Célhoz kötött** - tudjuk, hogy miért készül a modell, mit akarunk vele elérni, segít megérteni a problémát.
- **Válaszokat ad** - segít megtalálni a megoldást, mivel átlátjuk a problémakört és a modell csökkenti a bonyolultságot.

Domain modell



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája - Domain modell V4.9 | Unrestricted | Page 20

A **domain modell** (szakterületi modell) egy konkrét problémakört ír le és tesz érhetőbbé azok számára is, akik nem szakértők az adott területen.

Egy példa a domain modellre a Naprendszer ábrázolása - ennek segítségével ábrázolni tudjuk a bolygók és a Nap egymáshoz való viszonyát. Egy iskolában például remekül lehet használni arra, hogy bemutassuk a Naprendszer felépítését, működését, legfontosabb tulajdonságait.

A domain modell tulajdonságai

Segít megérteni, leírni a problémát

Közös nyelvet definiál

Absztrakt elemek a valóságból

Leírja a domain elemek közötti kapcsolatokat

Leírja a domain elemek viselkedését

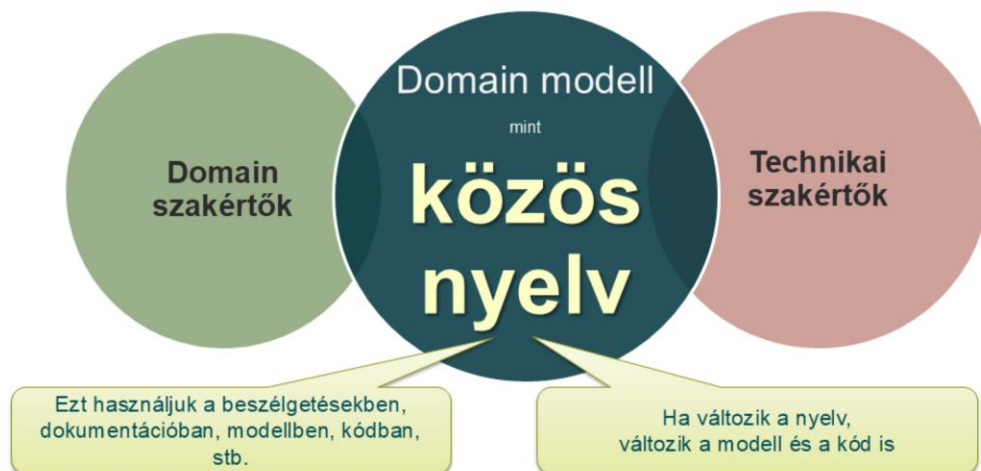


Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 21

A domain modell a szoftvertervezésben is hasonló tulajdonságokkal bír, mint a Naprendszer modellje.

Leegyszerűsíti a problémát, így a megoldásra lehet koncentrálni.



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse: evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 22

A domain modell biztosítja a közös nyelvet a **szakterület szakértői** és a **technikai szakértők** (pl. programozók) között, akik amúgy nem igazán tudnának szót érteni egymással. A domain modell kialakítása során megismerhetjük a szakterület nyelvezetét, zsargonját, a legfontosabb elemeket a szakterületen belül, ezeknek a viselkedését, a köztük levő kapcsolatokat, specializációkat.

A **közös nyelv** (és ezáltal maga a domain modell) kialakítása nem egyszerű, de rendkívül hasznos folyamat. Ha sikerül kialakítani ezt a nyelvet, akkor ezzel már gond nélkül meg tudjuk beszélni a domain szakértőkkel a konkrét követelményeket. Olyan megoldásokat tudunk kialakítani, amelyben otthonosan tudnak mozogni, és könnyen tudnak visszajelzést adni az elképzelt tervekre is.

Nagyon fontos, hogy ha kialakult ez a közös nyelv, akkor ezt használjuk a szoftver minden részében: a dokumentációban, a modellekben és a kódban is! Ha valahol ez nem egyezik, akkor a kerülőutak helyett érdemes megnézni, hogyan alakítható át a nyelv (és a domain modell) úgy, hogy feloldjuk a hiányosságot. Ha változik a kód nyelvezete, akkor a domain modellt is aktualizálni kell, és fordítva.

Közös nyelv

A közös nyelven könnyebben tudunk követelményeket, user story-kat megfogalmazni

„Szeretnék egy szobát, amiben lehet aludni, valamint egy szobát, ahol lehet főzni, egy másik szobát, ahol lehet enni, és a kettő legyen könnyen átjárható, és egy szobát, amelyben lehet ruhákat tárolni, ...”

„Szeretnék egy 60 m²-es, világos, amerikai konyhás, gardróbos lakást”

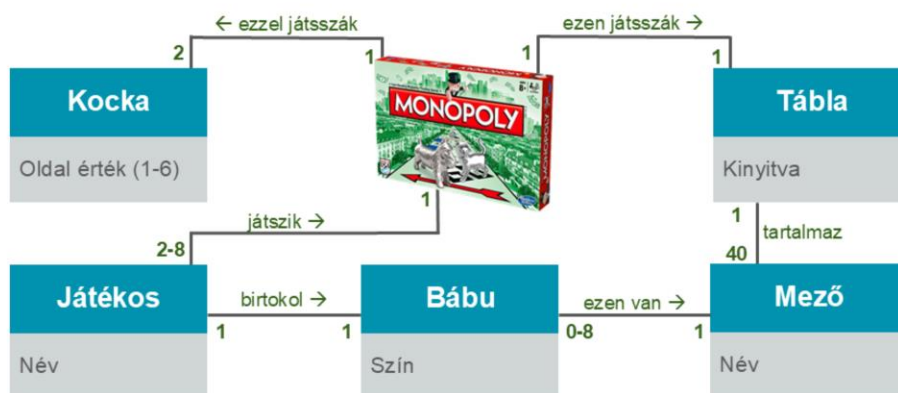


Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse: evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 23

A közös nyelv kialakításával és használatával nem csak a félreértéseket tudjuk elkerülni, hanem sokkal rövidebben és egyszerűbben tudunk megfogalmazni rajta követelményeket, feladatokat.

Ha azt érezzük, hogy a rendszerről csak nagyon körülményesen tudunk beszélni, akkor érdemes megnézni, hogy hogyan írják le ugyanezt a problémát az adott domain szakértői. Ha lehet, építsük be a domain modellbe az általuk használt elemeket.



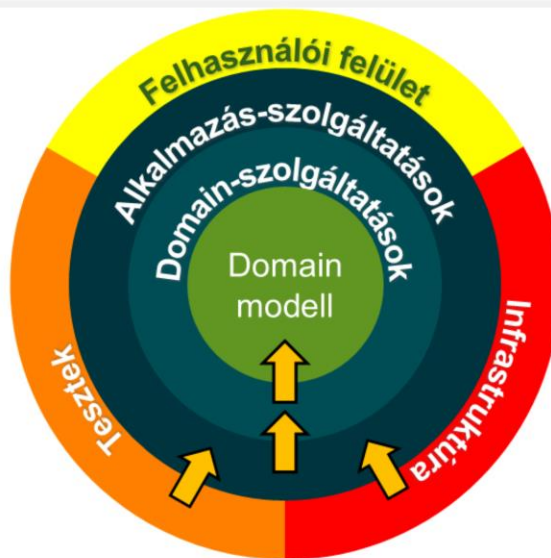
Példa egy domain modellre: Monopoly

- Nem írja le a szabályokat.
- A domain elemekkel le tudjuk írni a követelményeket.

Domain-vezérelt tervezés



A rétegelt architektúrában minden réteg függ az alatta levőtől, anélkül nem tud működni. Bármilyen változás a lentebb lévő rétegben az összes felette lévő réteg megváltozásával jár. És minden réteg függ az infrastruktúrától is, vagyis ha itt valami változik, az szintén az egész alkalmazásra jelentős kihatással jár.



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
 evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

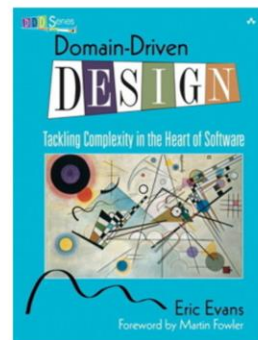
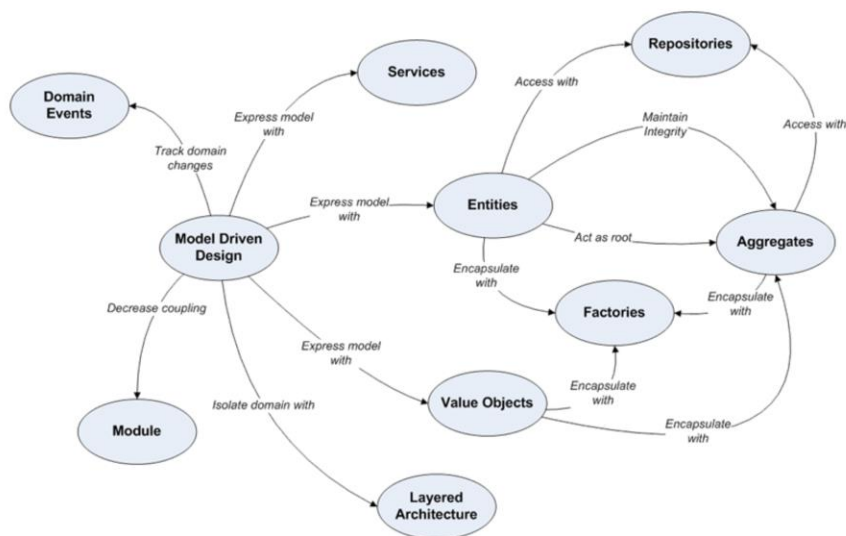
© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 27

A hagyma (onion) architektúra segíthet abban, hogy olyan alkalmazásokat tervezzünk, amelyek jobban támogatják a tesztelhetőséget, a karbantarthatóságot.

Az alkalmazást több rétegre bontjuk, amelyek úgy veszik körül egymást, mint a hagyma különböző rétegei.

- **Domain modell:** a rendszer magja. Itt található a szakterület alapvető koncepcióit, fogalmait leíró entitások.
- **Domain-szolgáltatások:** itt definiáljuk a szakterület folyamatait.
- **Alkalmazás-szolgáltatások:** itt kerülnek megvalósításra az alkalmazással kapcsolatos folyamatok, a használati esetek.
- **Felhasználói felület:** lehet WinForms, WPF vagy bármilyen web-es technológiával kialakítva.
- **Infrastruktúra:** az adatbázis, a levélküldés, minden külső szolgáltatás
- **Tesztek:** nem az alkalmazás működtetéséhez kellene, hanem annak megállapításához, hogy a kívánt működést valósítottuk-e meg.

A rétegek szigorúan meghatározott kapcsolatban vannak egymással: a **belső réteg nem hivatkozhat a kívül levőkre**, azokról semmit nem tudhat. Ezzel tudjuk biztosítani, hogy bármely külső réteg megváltoztatható, lecserélhető anélkül, hogy a belső rétegeket bármilyen módon is meg kellene változtatnunk.



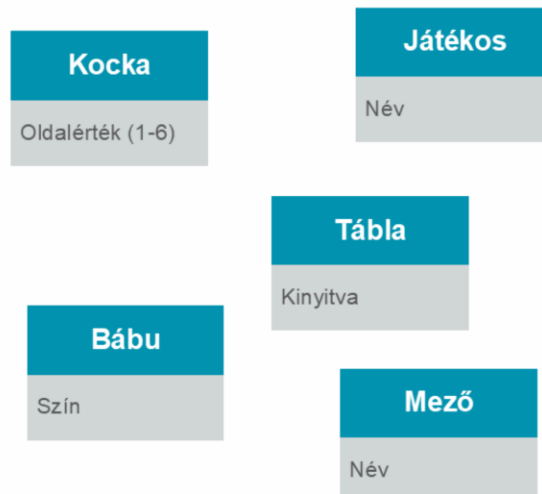
Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 28

A domain modell az alapja annak az **Eric Evans** által megalkotott módszernek, amit **Domain Driven Design**-nak, vagyis domain-vezérelt tervezésnek nevezünk.

Entitás (entity)

- Elsődleges szakterületi fogalom
- Egyedi azonosítóval rendelkezik
- Azonosítója állandó
- Tulajdonságai változhatnak élettartama alatt
- Két entitás azonos, ha azonosítóiik megegyeznek



Értékelem (value object)

- Entitások tulajdonsága
- Nincs egyedi azonosítója
- Értéke állandó (immutable)
- Két értékelem azonos, ha minden tulajdonságuk (értékük) megegyezik
- Könnyű létrehozni és megszüntetni

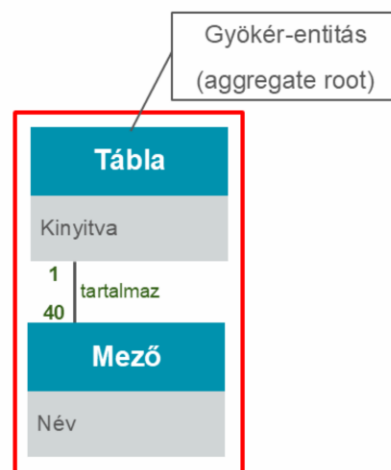


Oldalérték: egyszerű számérték.

Név: szerkezete lehet összetett is, ha a vezetéknév és a keresztnév kombinációját tekintjük egy egységnek.

Aggregátum (aggregate)

- Objektumok csoportja, amelyek adatváltozási szempontból egy egységet képeznek
- Egyetlen gyökér-entitással rendelkezik
- Csak a gyökér-entitás hivatkozható közvetlenül kívülről
- A benne tárolt entitások azonosítója lokálisan egyedi
- A benne tárolt értékek módosítása csak a gyökér-entitáson keresztül indítható
- Az aggregátumon belül a konzisztenciát szinkron kell biztosítani
- Az aggregátumok között az adatok módosítása aszinkron történik



Szolgáltatás (service)

- Szakterületi művelet, amely nem tartozik objektumhoz
- Más domain objektumokon végez módosításokat
- A művelet állapotmentes (stateless)

JátékmenetService

```
StartJátékmenet(...) : Játékmenet  
PauseJátékmenet(Játékmenet)  
StopJátékmenet(Játékmenet)
```


Gyár (factory)

- Új entitások és aggregátumok létrehozására szolgál
- A létrehozás atomi művelet
- Hiba esetén kivétel keletkezik
- Az eredmény állapota mindig jól definiált
- Lehetőség szerint absztrakt objektumot ad vissza
- Aggregátum gyökér-entitása is lehet gyár

TáblaFactory

CreateTábla(...) : Tábla

JátékmenetFactory

CreateJátékmenet(...) : Játékmenet

Tároló (repository)

- Létező entitások és aggregátumok visszaadására szolgál
- Elrejtí a tárolás és a visszanyerés részleteit a domain objektumai elől
- Adott típusú entitás memóriában tárolt gyűjteményeként viselkedik
- Műveletek:
 - Hozzáadás
 - Törlés
 - Visszaadás azonosító alapján
 - Keresés különböző kritériumok szerint

TáblaRepository

GetTábla(id) : Tábla

JátékosRepository

AddJátékos(...) : Játékos
GetJátékos(id) : Játékos
DeleteJátékos(id)

JátékmenetRepository

AddJátékmenet(...) : Játékmenet
GetJátékmenet(id) : Játékmenet
GetLastJátékmenet() : Játékmenet
GetSavedJátékmenetek() : Játékmenet[]
DeleteJátékmenet(id)

Domain esemény (domain event)

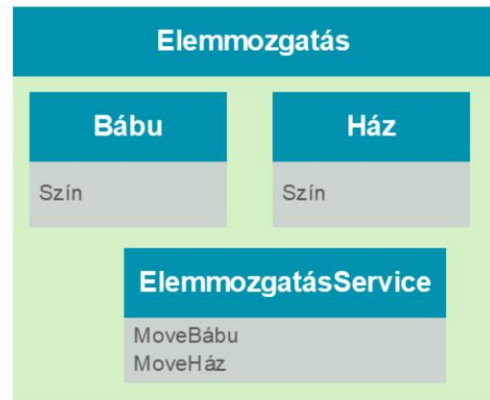
- Szakterületi esemény
- Elosztott rendszerekben biztosítja az entitások konzisztenciáját
- Értéke állandó (immutable)
- Többször feldolgozható, több szereplő által

BankPénzeElfogyott

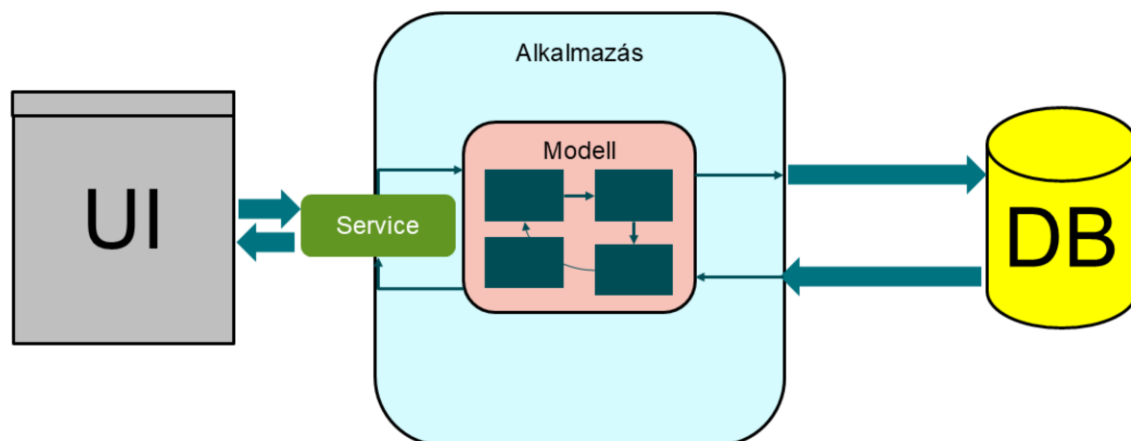
Forrás : Bank
Időpont : Dátum + idő

Modul (module)

- Összetartozó fogalmak rendszerezése
- Csökkenti a rendszer komplexitását
- Jól definiált interfésszel rendelkezik
- Laza csatolás a modulok között
- Erős kohézió a modulon belül
 - Kommunikációs: azonos adatokon dolgoznak a modul részei
 - Funkcionális: egy jól meghatározott feladaton dolgoznak együtt a modul részei



CRUD / CQRS



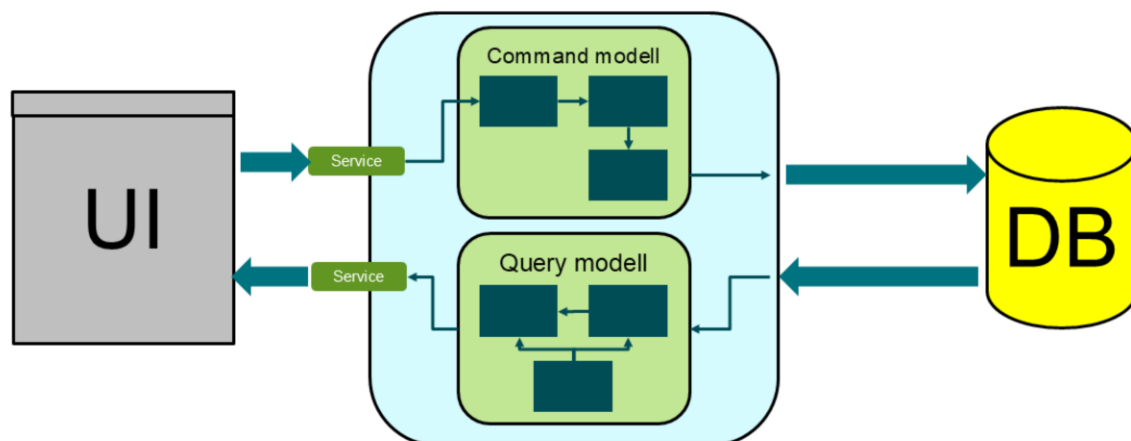
Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 38

A DDD-vel kapcsolatban egy korábban sokat használt tervezési minta a **CRUD**. Itt az alkalmazás a felhasználói felület (UI) és az adattárolás, az adatbázis között végez feladatokat. A UI felé egy szolgáltatás komponens segíthet a kapcsolati függőséget csökkenteni.

Az objektumok életciklusában a létrehozás, kiolvasás, módosítás és törlés műveletek fordulnak elő. A CRUD módszerben a **központi elem az adatbázis**, hiszen az objektumok tárolásával kapcsolatos fogalmakra, műveletekre helyezi a hangsúlyt.

Ahogy a modell kezd egyre komplexebb lenni, az egyes tárolási műveleteknek egyre többször kell egyszerre több objektumon műveleteket végezni. Ez pedig exponenciális **komplexitásnövekedést** jelent.



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11. Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 39

A korábbi megoldás hibáinak kiküszöbölésére alakult ki a **CQRS** módszer. Itt az alkalmazás környezete ugyanolyan, a belseje azonban más. Az ötlet alapja az, hogy az adatbázis olvasása nem jár semmilyen módosítással, míg a többi művelet megváltoztatja a tárolt adatot.

Ezért válasszuk szét a domain modellünket két külön részre:

- A **Command modell** felel az adatok módosításáért, mert ez a művelet mindig valamilyen explicit parancsra megy végbe,
- A **Query modell**ben végezzük el a lekérdezések kezelését. Ebben az objektumok adattartalma illeszkedhet az adott lekérdezéshez, nem kell feltétlenül mindig minden részlelemet kiolvasni az adatbázisból.

Az adatutakat is sokkal jobban szét tudjuk választani, ezáltal az egyes részek optimalizálása is könnyebbé válik. Gondoljunk csak bele, hogy egy átlagos rendszerben milyen a lekérdezések és a módosítások aránya. Így az esetek túlnyomó részében használt lekérdezéseket célzottan tudjuk gyorsítani, míg a módosításokhoz tartozó üzleti logika így biztosan nem lesz érintett. Ez hatalmas rizikóktól szabadítja meg a rendszerfejlesztést!

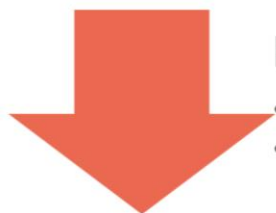
Link: <https://martinfowler.com/bliki/CQRS.html>

Összefoglalás



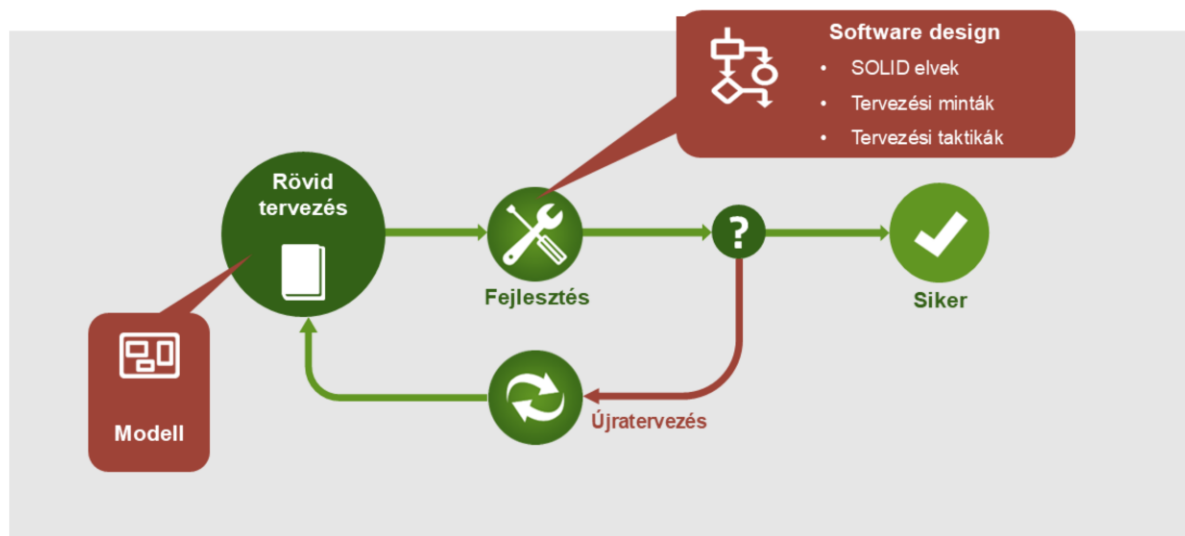
Előnyök

- Megkönnyíti a kommunikációt
- A valódi felhasználóra koncentrál
- Moduláris és eléggé független



Hátrányok

- Erős domain tudást feltételez
- Nem alkalmas erősen technikai projektekhez



Jelen alkotás szerzői jogi védelem alatt áll. A felhasználáshoz szükséges engedélyért keresse:
evosoft Hungary Kft. H-1117 Budapest, Magyar tudósok körútja 11., Tel: +36 (1) 38-16400

© evosoft Hungary Kft. 2026 | Nagy rendszerek fejlesztésének technológiája – Domain modell V4.9 | Unrestricted | Page 42

Az év folyamán, a következő előadásokon lesz még szó hasznos technikákról, amelyek segítik az agilis szoftvertervezést.

evosoft Hungary Kft.

Magyar tudósok körútja 11.

H-1117 Budapest

Telefon: +36 (1) 3816-400

Telefax: +36 (1) 3816-101

Web: www.evosoft.com

Előadó

Békés Tamás

Vezető szoftvermérnök / szoftver architekt

E-Mail: evo_uninik_hu@evosoft.com



Your powerful partner

www.evosoft.com